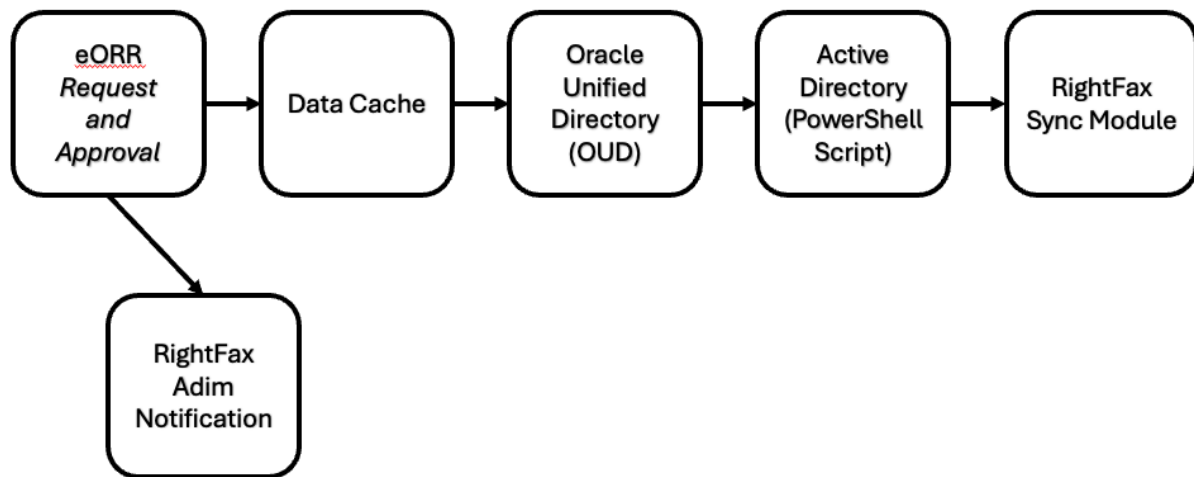


RightFax Provisioning Automation Project

Project Mission

Automate account approvals and user configuration to streamline the creation process and reduce manual workload.

Provisioning Pipeline Overview



Full provisioning pipeline from request to account.

eORR Role Request

The RightFax account approval matrix is consolidated into an eORR role data spreadsheet for system integration. Users obtain fax line access by requesting the appropriate role within eORR. Once confirmed by designated approvers, the automated pipeline processes the request and assigns the role to the user in Active Directory (AD).

PowerShell Script

Upon the addition of the FSU_FAX role in AD, a PowerShell script assigns the user to the corresponding RightFax-mapped AD group. If the target group does not exist, the script automatically initializes it. Users may belong to multiple synced AD groups, allowing for several eORR requests per individual. This synchronization script executes on an hourly basis.

RightFax Sync Module

The synchronization module monitors Active Directory at a preconfigured frequency to identify new users and retrieve their data via XML. Utilizing an XSLT transformation alongside a designated model user, the module maps source attributes to automatically configure and initialize the new accounts.

RightFax Sync Module Configuration

This section details the configuration that is necessary for mapping the AD groups with RightFax groups.

1. Service Account Identity

The first required setting is the **Windows Service Account**. If the module is synchronizing with Active Directory, this account **must be a member of the AD Domain Users group**. This setting provides the necessary network authority for the module to read the external data source.

2. Source and Destination Definitions

The data source should be defined as our Active Directory account. There is a 1-to-1 relationship AD group to data destination. Each destination will use the RightFax database, this is strictly to allow unique model users for each mapping.

3. Destination Database Settings

The RightFax User Templates (model users) are specific RightFax accounts designated as a model for new user creation. This is configured because new users inherit all settings such as group membership, permissions, and delegate access associated with this model user. Enable the **"Attempt to map onto existing RightFax users"** setting. This ensures the module checks for matches in the existing database before creating a new account, which prevents the creation of duplicate users.

4. Source Filter Settings

Filters are the mechanism that determines which users are pulled into the sync pipeline. Every filter must include a domain and the AD group in the **Included Groups** (entering the organizational unit (OU) will reduce the number of AD group options that populate). Filters should include **Excluded Groups**, which are configured to create a priority system. This prevents user configurations from being overwritten if a user belongs to multiple AD groups that are targeted by different mappings. This means additional groups appended to the user will require manual configuration by the RightFax Admin (e.g. delegate access).

5. Output and Property Collections

The **Output** settings define exactly what data is extracted from the source and how to handle users who are no longer found in the directory. The **Property Collection** is a list of specific text

attributes (e.g displayName, mail, etc.) that the module monitors in the source. The **Property Picker** is used to manually select these attributes from the AD schema. This is configured to ensure the module only tracks the specific data points required for your configurations. To manage users who are **no longer in the sync group**, the setting '**Treat users... as if they had been deleted**' must be enabled. To ensure these users are deprovisioned (unmapped) without losing their fax history, this should be paired with the '**Never delete users**' safety setting. Note that the '**Disabled Users**' section is a separate configuration used specifically to **disable** RightFax accounts for users who remain in the sync group but have been disabled in Active Directory.

6. Transform File

Each mapping (destination) is assigned an **XSLT-based transform file**. While the property collection identifies the required attributes from AD, the transform file contains the logic that labels and assigns those attributes to the correct fields in the RightFax user profile.

7. Sync Frequency

Schedule: The mapping defines the **frequency** of the synchronization, which can be set to run daily at a specific time or recurring every specified number of hours.

Transform File

```
<!--
AD Data XML -> Transform File (XMTL) -> Cleaned Data (XMT) -> RightFax Service (Program) -> RightFax Database (SQL Database)
-->

<!-- Configures file and -->
<?xml version="1.0"?>
<xsl:stylesheet
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform" <!-- Links to standard XML namespace for XSL -->
  <!-- Defines the RightFax namespace (for declaring that source "rf:" tags are to become destination "urn:rightfax-sync-schema" tags) -->
  <!-- Signals program to serialize any untagged attributes under the uniform resource name "rightfax-sync-schema" (XSD)-->
  <!-- Removes any "rf" prefixes to clean data for RightFax Service -->

  <xsl:output method="xml" indent="yes"/> <!-- makes output XML -->

  <!-- Copy all nodes and attributes. -->
  <xsl:template match="/" | @* | node()*> <!-- Signals to scan & copy all incoming data -->
    <xsl:copy>
      <xsl:apply-templates select="@* | node()"/>
    </xsl:copy>
  </xsl:template>

  <!-- "userID" is a required attribute -->
  <xsl:template match="rf:attr[@name='sAMAccountName']"><!-- rule (template) defined for catching sAMAccountName -->
    <attr name="UserID"> <!-- literal element with name attribute for output -->
      <xsl:apply-templates select=""/> <!-- retrieves the value and applies template -->
    </attr>
  </xsl:template>

  <xsl:template match="rf:attr[@name='objectSid']">
    <attr name="AssociatedNTAccountSID">
      <xsl:apply-templates select=""/>
    </attr>
    <attr name="RequiresNTAuth">
      <value> </value>
    </attr>
  </xsl:template>

  <xsl:template match="rf:attr[@name='facsimileTelephoneNumber']">
    <attr name="RouteCode">
      <value>
        <!-- Get the last 10 digits of fax number, ignoring various special characters that might
        be found in phone numbers. If result is Not a Number then use default route code instead.
        TAKING TEN DIGITS WILL ALWAYS RESULT IN THE ROUTE CODE BEING 100-->
        <xsl:variable name="cleanNumber" select="translate(node(), ' .-()','')"/>
        <xsl:variable name="routeCode" select="substring($cleanNumber, string-length($cleanNumber)-9, 10)"/>
        <xsl:choose>
          <xsl:when test="string(number($routeCode))='NaN'"> </xsl:when>
          <xsl:otherwise><xsl:value-of select="$routeCode"/></xsl:otherwise>
        </xsl:choose>
      </value>
    </attr>
  </xsl:template>
```